

# Understanding Agent toolings

Maurice Dalton

DISSC / Tobin Center

## Goals for today

1. Understand how Agents use tools
2. Understand how you can create custom tools for agents
3. Provide enough information so you can understand what third party tools are doing
4. Provide a working example

Motivations

# AI Agents for Economic Research

August 2025 Update of “General  
AI for Economic Research: Use Cases  
and Implications for Economists”  
Published in *Journal of Economic  
Literature* 61(4)\*

# Towards Agentic chatbots

| Table 1<br>Primary Usefulness of LLM Model Types for Research Tasks |                                                                           |                                                                    |                                                                            |
|---------------------------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------|
| Research Category                                                   | Traditional LLMs                                                          | Reasoning Models                                                   | Agentic Chatbots                                                           |
| Ideation & Feedback                                                 | Good for initial brainstorming.                                           | <b>Best for structured feedback and identifying logical flaws.</b> | Actively scans literature for novelty and grounding.                       |
| Writing                                                             | <b>Excellent for drafting, summarizing, and rephrasing existing text.</b> | Ensures logical flow in complex arguments.                         | Incorporates real-time web information.                                    |
| Background Research                                                 | <i>Shortcoming: No live web access.</i>                                   | Synthesizes info from provided texts.                              | <b>Best for live web searches and up-to-date literature reviews.</b>       |
| Coding                                                              | Generates basic code snippets.                                            | <b>Excellent for writing and debugging complex algorithms.</b>     | <b>Executes code, tests hypotheses, and interacts with data files.</b>     |
| Data Analysis                                                       | <i>Shortcoming: Cannot execute code.</i>                                  | Helps interpret data and suggest approaches.                       | <b>Best for end-to-end analysis: data cleaning, coding, visualization.</b> |
| Math                                                                | <i>Shortcoming: Unreliable for complex math</i>                           | <b>Solves multi-step problems and formal</b>                       | Can leverage external computational tools to                               |

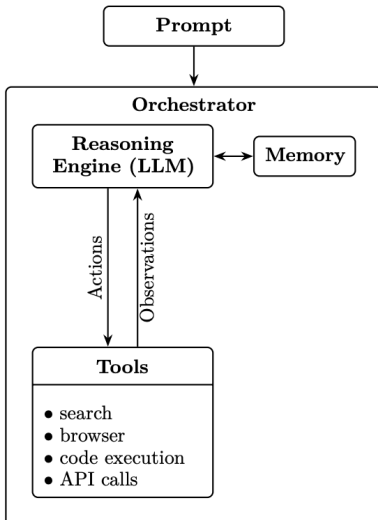
## Demystifying AI Agents

What are these new tools?

AI Agents are mostly autonomous systems built on LLMs by giving them access to tools that can:

- ▶ Manage context, e.g. using memory to keep track of previous steps
- ▶ Create detailed plans
- ▶ Use various tools (e.g. web search, code execution, data analysis, write reports) to decide to add context
- ▶ Execute multi-step tasks
- ▶ Orchestrate tasks between specialized agents

What does using an Agentic Chatbot look like?



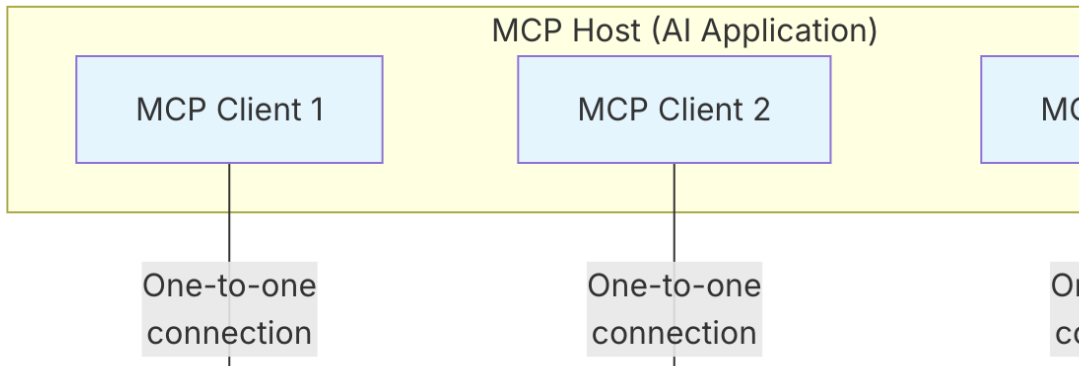
## What are tools in the Agent context?

- ▶ Model **Context** Protocol (MCP) were developed by anthropic to provide additional context to the LLM
- ▶ Expose additional information for the LLM to use
- ▶ LLMs discover what tools are available and decide how to use them
  - ▶ Tools should focus on high-level capability that aligns with a user's intent, rather than a thin wrapper for an API
  - ▶ The metadata associated with each tool—its name, title, description, and inputSchema—forms a “semantic contract” which impact how the LLM uses the tool
- ▶ <https://modelcontextprotocol.io/docs/getting-started/intro> the doc site is an excellent place to start and much of the material is pulled from this source

## MCP: Architecture for AI Agents

MCP are made up three parts

1. MCP Host, this is the interface; e.g. could be a chat interface or code
2. MCP client: they interact between the LLM and the tools.
3. MCP server: Tool provides actions and descriptions of capabilities





## Tools calling flow

When you submit a query:

1. The client gets the list of available tools from the server
2. Your query is sent to the LLM, lets say Claude, along with tool descriptions
3. Claude decides which tools (if any) to use
4. The client executes any requested tool calls through the server
5. Results are sent back to Claude
6. Claude provides a natural language response
7. The response is displayed to you

Source: MCP

# MCP server

Servers provide functionality through three building blocks:

| Feature   | Explanation                                                                                                                                                                             | Examples                                                       | Who controls it |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|-----------------|
| Tools     | Functions that your LLM can actively call, and decides when to use them based on user requests. Tools can write to databases, call external APIs, modify files, or trigger other logic. | Search flights<br>Send messages<br>Create calendar events      | Model           |
| Resources | Passive data sources that provide read-only access to information for context, such as file contents, database schemas, or API documentation.                                           | Retrieve documents<br>Access knowledge bases<br>Read calendars | Application     |
| Prompts   | Pre-built instruction templates that tell the model to respond with                                                                                                                     | Plan a vacation                                                | User            |

## Pophive example, chatting with your data



POPHIVE

Yale SCHOOL OF PUBLIC HEALTH

HOME

*Population Health Information and Visualization Exchange*

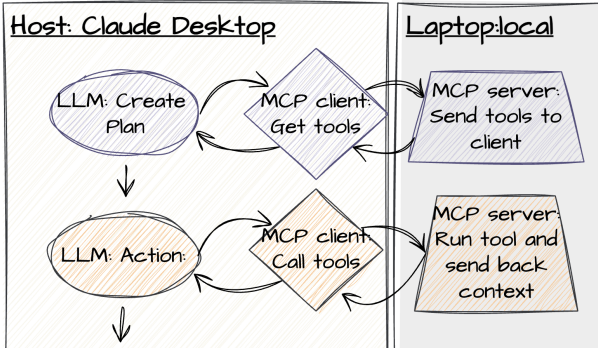
We're on a mission to reimagine health data for all.

# Track health trends in your community.

## Pophive example architecture

USER:

Provide correlation between obesity and diabtities by state via a heatmap and compare to other national trends



## Creating a local MCP server

- ▶ While the ultimate goal is to explore setting an agentic chatbot on a website, we will build a local instance to show how it works
- ▶ Will rely on the MCP SDK python library - <https://github.com/modelcontextprotocol/python-sdk>

## Initialize MCP server

```
import os
import json
import base64
from io import BytesIO
from pathlib import Path
from typing import List, Dict, Any, Optional

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

from mcp.server.fastmcp import FastMCP

# Initialize the MCP server
mcp = FastMCP("DataVisualizationServer")
```

## Initializing tools

- ▶ Next you create function with the `@MCP.tool()` decorator
- ▶ The name of the tool and the description are used by the LLM to decide how to use the tool
- ▶ Note that the server has access to the data found by `DATA_DIR`, you could have other connections to data here

```
@mcp.tool()
def list_data_files() -> str:
    """Lists all available data files in the data directory wi
```

## Tooling logic

- ▶ logic looks like a regular python function with context sent back to llm via return statement

```
try:
    if not DATA_DIR.exists():
        return f"Error: Data directory '{DATA_DIR}' does not exist."

    files = []
    for file_path in DATA_DIR.glob("*"):
        if file_path.is_file():
            stat = file_path.stat()
            files.append({
                "name": file_path.name,
                "size_bytes": stat.st_size,
                "size_mb": round(stat.st_size / (1024 * 1024), 2),
                "extension": file_path.suffix
            })

    if not files:
        return "No data files found in the data directory."

    # Format as a nice table
    result = "Available Data Files:\n\n"
    result += "| Filename | Size (MB) | Type |\n"
    result += "|-----|-----|-----|\n"
```



## Tools

```
@mcp.tool()  
def list_data_files() -> str: ...
```

```
@mcp.tool()  
def describe_dataset(filename: str) -> str: ...
```

```
@mcp.tool()  
def generate_correlation_plot(filename: str, plot_type: str = "scatter") -> str: ...
```

```
@mcp.tool()  
def list_generated_images() -> str: ...
```

## Everything the MCP server returns consumes tokens

- ▶ returning a native figure was using up all my tokens
- ▶ save to path instead

```
# Convert to base64
buffer = BytesIO()
plt.savefig(buffer, format='png', dpi=150, bbox_inches='tight')
buffer.seek(0)
image_base64 = base64.b64encode(buffer.read()).decode('utf-8')
buffer.close()

# Save image to file instead of returning base64
output_filename = f"{filename.split('.')[0]}_{plot_type}_correlation.png"
output_path = OUTPUT_DIR / output_filename
plt.savefig(output_path, format='png', dpi=150, bbox_inches='tight')
plt.close()

return f"Generated {plot_type} plot for {filename}. Image data: data:image/png;base64,{image_base64}"
return f"Generated {plot_type} plot for {filename}. Image saved to: {output_path}"

except Exception as e:
    return f"Error generating correlation plot: {str(e)}"
```

## Lastly, give the host access to the tools

Since this example uses Claude Desktop

1. update the `~/Library/Application Support/Claude/claude_desktop_config.json` file with the following
2. This gives claude access to a new tool

```
{
  "mcpServers": {
    "data-visualization": {
      "command": "$HOME/git-pub/dissc-agent-tooling/tasks/data-visualization-mcp/venv/bin/python",
      "args": [
        "$HOME/git-pub/dissc-agent-tooling/tasks/data-visualization-mcp/data_viz_server.py"
      ],
      "cwd": "$HOME/git-pub/dissc-agent-tooling/tasks/data-visualization-mcp",
      "env": {
        "DATA_DIR": "$HOME/git-pub/dissc-agent-tooling/data",
        "OUTPUT_DIR": "$HOME/git-pub/dissc-agent-tooling/output/images"
      }
    }
  }
}
```

## MCP for a wider audience

- ▶ Worth mentioning that MCP servers can be hosted
- ▶ Additional security and cost need to be considered

## Towards modular agents

- ▶ Developing tools for an LLM to use is the first step in developing modular agents
- ▶ Encourage people to think about breaking down tasks into smaller components so both tools and agents can be reused
- ▶ Agent2Agent communication is one way to achieve this, in that it allows you to abstract to the agent level allowing agent to discover what each agent can do